

Introduction to Generative Adversarial Networks

Understanding GANs with Python Implementation

Jiaping(Olivia) Liu

2025-10-10

Table of contents

0.1	What are GANs?	1
0.2	Visualization Demo: 2D Distribution Learning Progression	2
0.3	GAN Architecture	2
0.4	Mathematical Foundation	2
0.5	The GAN Game Theory	3
0.6	Applications	4
0.7	Training Challenges	4
0.8	GAN Variants	5
0.9	Conditional GAN (cGAN)	5
0.10	How Training & Generation Works	6
0.11	Conditional GAN Objective	6
0.12	Applications of cGAN	6
0.13	Example: MNIST Conditional Generation	7
0.14	Key Advantages	7
0.15	Performance Evaluation	7
0.16	Tips for Successful GAN Training	7
0.17	Resources and Next Steps	8
0.18	Key Takeaways	8
1	Appendix	8
1.1	Simple GAN Implementation	8
1.2	Training Process Demo	9
1.3	Code Structure for Practice	10

0.1 What are GANs?

- **Generative Adversarial Networks** - Deep learning architecture
- Introduced by Ian Goodfellow in 2014
- Two neural networks competing against each other
- **Generator**: Creates fake data
- **Discriminator**: Distinguishes real from fake data
- Training through adversarial process

0.2 Visualization Demo: 2D Distribution Learning Progression

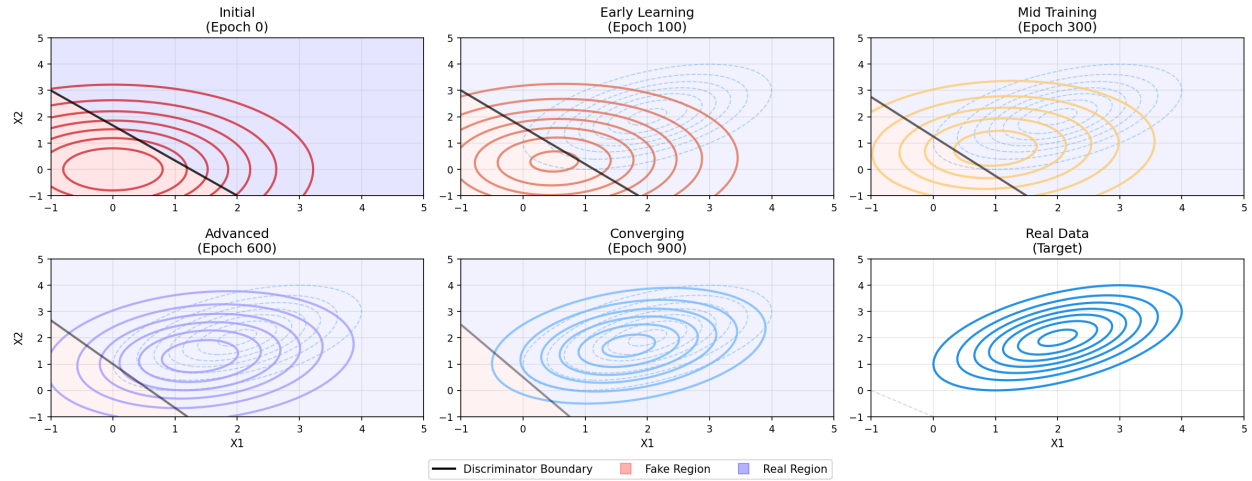


Figure 1: GAN Learning Progression

0.3 GAN Architecture

0.3.1 Generator Network

- Takes random noise as input
- Transforms noise into realistic data
- Goal: Fool the discriminator
- Trained to minimize detection

0.3.2 Discriminator Network

- Takes real and fake data
- Outputs probability (real vs fake)
- Goal: Correctly classify data
- Trained to maximize accuracy

0.4 Mathematical Foundation

The GAN objective function:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Where:

- G: Generator network
- D: Discriminator network

- x : Real data samples
- z : Random noise vector

0.5 The GAN Game Theory

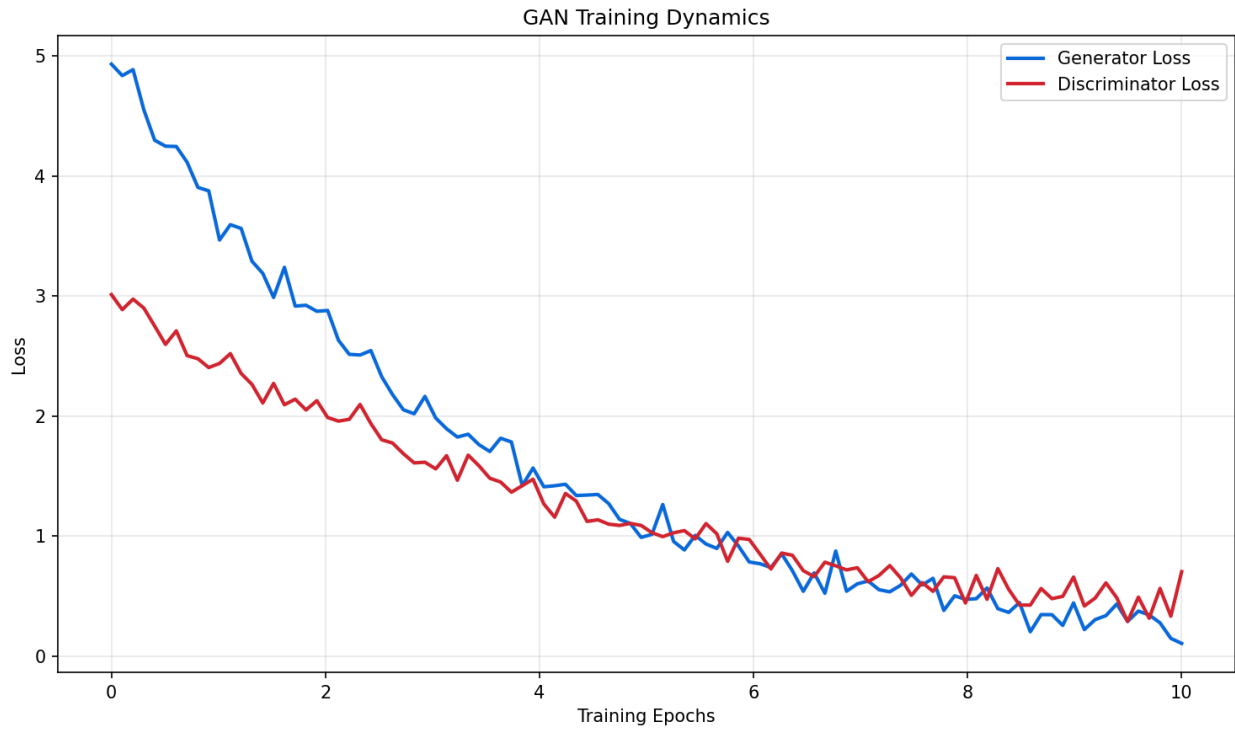


Figure 2: GAN Training Dynamics

0.6 Applications

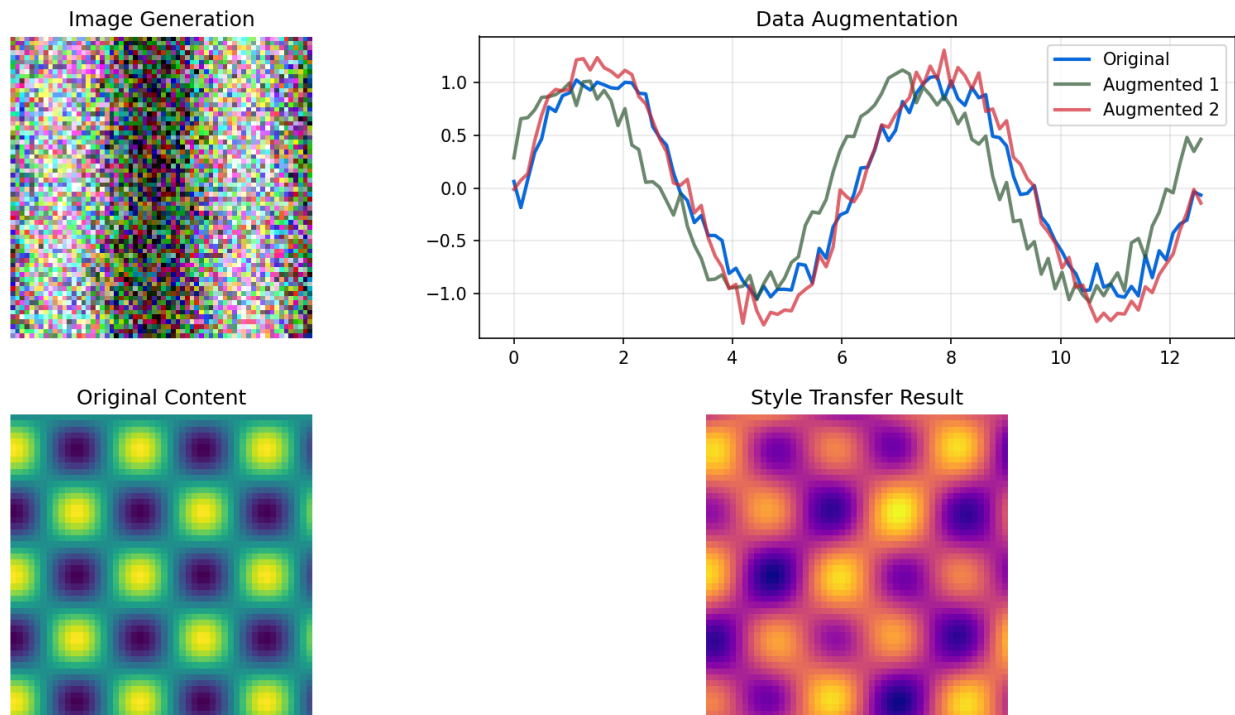


Figure 3: GAN Applications

0.7 Training Challenges

- **Mode Collapse:** Generator produces limited variety
- **Training Instability:** Oscillating losses
- **Vanishing Gradients:** Poor gradient flow
- **Nash Equilibrium:** Difficult to reach balance

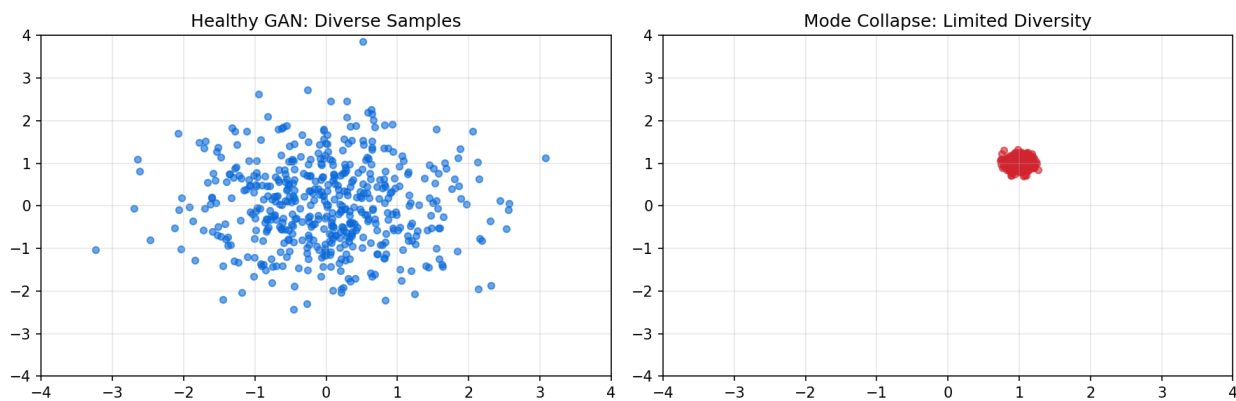


Figure 4: Mode Collapse Demonstration

0.8 GAN Variants

0.8.1 Popular Architectures

- **DCGAN**: Deep Convolutional GAN
- **StyleGAN**: Style-based Generator
- **CycleGAN**: Unpaired Image Translation
- **Pix2Pix**: Paired Image Translation

0.8.2 Improved Training

- **WGAN**: Wasserstein GAN
- **LSGAN**: Least Squares GAN
- **Progressive GAN**: Progressive Growing
- **Self-Attention GAN**: SAGAN

0.9 Conditional GAN (cGAN)

0.9.1 What is Conditional GAN?

- **Extension of vanilla GAN** with conditional information
- Both Generator and Discriminator receive **additional input** (labels, attributes, etc.)
- Enables **controlled generation** of specific outputs
- First introduced by Mirza & Osindero (2014)

0.9.2 Architecture & Training

Generator: $G(z, y)$

- Input: Random noise z + condition y
- Output: Fake sample x_{fake}
- Learns: $P(x|y)$ distribution

Discriminator: $D(x, y)$

- Input: Sample x + condition y
- Output: Probability of being real
- Learns to distinguish real/fake given condition

0.10 How Training & Generation Works

During Training:

- Real data comes with labels: (x_{real}, y_{real})
- Example: (image of cat, label = “cat”), (image of dog, label = “dog”)
- Condition y is **given from the dataset**
- Generator learns: “When $y = \text{cat}$, generate cat images”

During Generation (Inference):

- You **manually specify** the condition y
- Want a cat? Set $y = \text{“cat”}$ + random noise $z \rightarrow G(z, y)$ generates cat
- Want a dog? Set $y = \text{“dog”}$ + random noise $z \rightarrow G(z, y)$ generates dog
- **Full control** over what to generate!

0.11 Conditional GAN Objective

Modified objective function with conditioning:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x|y)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z|y)))]$$

Where y represents the conditional information (class labels, attributes, text descriptions, etc.)

0.12 Applications of cGAN

1. **Text-to-Image:** Generate images from text descriptions
2. **Class-Conditional Generation:** Generate specific object classes (e.g., MNIST digits 0-9)
3. **Image-to-Image Translation:** Pix2Pix - edges to photos, sketches to realistic images
4. **Super-Resolution:** Generate high-res images conditioned on low-res input
5. **Style Transfer:** Transfer artistic style while preserving content
6. **Data Augmentation:** Generate labeled synthetic data for training

0.13 Example: MNIST Conditional Generation

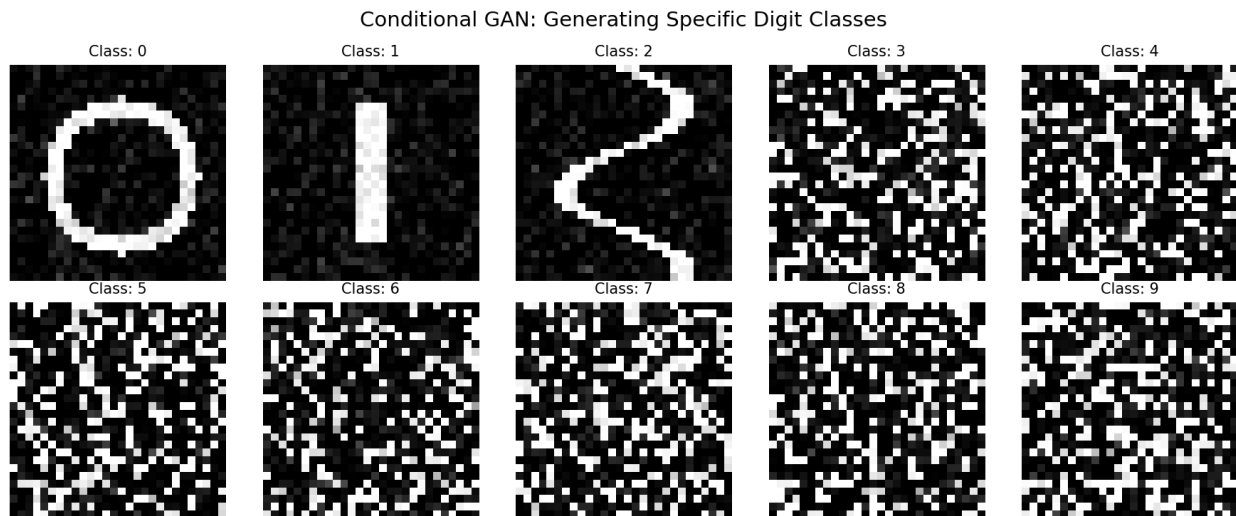


Figure 5: Conditional GAN Example

0.14 Key Advantages

- **Control:** Generate samples with specific attributes
- **Flexibility:** Condition on various types of information
- **Quality:** Often better quality than vanilla GAN
- **Interpretability:** Easier to understand what's being generated
- **Practical:** More applicable to real-world tasks

0.15 Performance Evaluation

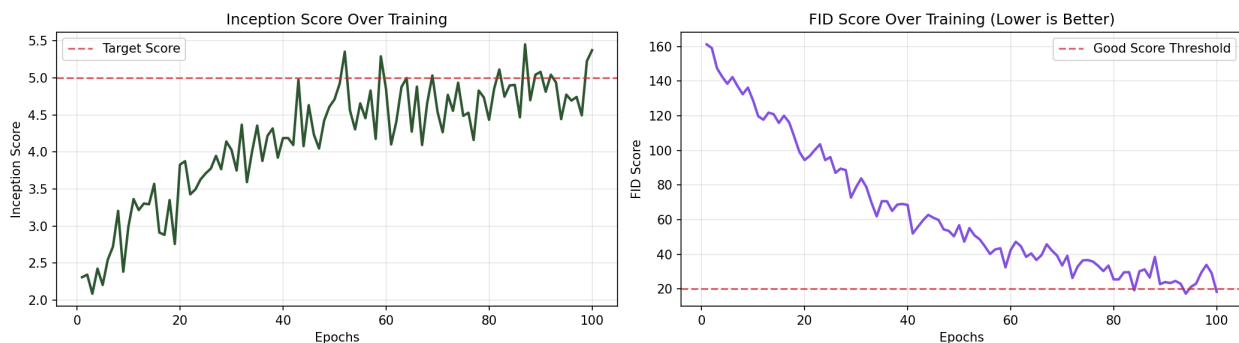


Figure 6: Performance Metrics

0.16 Tips for Successful GAN Training

1. **Balance the Networks:** Similar capacity for G and D

2. **Learning Rate Scheduling:** Careful tuning required
3. **Batch Normalization:** Stabilizes training
4. **Leaky ReLU:** Better gradients than standard ReLU
5. **Feature Matching:** Alternative loss functions
6. **Historical Averaging:** Prevent oscillations
7. **Monitor Metrics:** Track IS, FID, visual quality

0.17 Resources and Next Steps

0.17.1 Further Reading

- Original GAN Paper (Goodfellow et al., 2014)
- DCGAN Paper (Radford et al., 2015)
- Progressive GAN (Karras et al., 2017)
- StyleGAN Series (Karras et al.)

0.17.2 Practical Implementation

- PyTorch GAN tutorials
 - TensorFlow GAN library
 - Papers with Code implementations
 - Kaggle competitions
-

0.18 Key Takeaways

- GANs learn through adversarial competition
- Generator creates, Discriminator evaluates
- Applications span from art to data augmentation
- Training requires careful balance and monitoring

0.18.1 Questions & Discussion

1 Appendix

1.1 Simple GAN Implementation


```

import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np

class Generator(nn.Module):
    def __init__(self, latent_dim, output_dim):
        super(Generator, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Linear(256, output_dim),
            nn.Tanh()
        )

    def forward(self, z):
        return self.model(z)

class Discriminator(nn.Module):
    def __init__(self, input_dim):
        super(Discriminator, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 128),
            nn.LeakyReLU(0.2),
            nn.Linear(128, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.model(x)

```

1.2 Training Process Demo

```

# Initialize networks
latent_dim = 100
data_dim = 2

generator = Generator(latent_dim, data_dim)
discriminator = Discriminator(data_dim)

# Optimizers

```

```

lr = 0.0002
g_optimizer = optim.Adam(generator.parameters(), lr=lr)
d_optimizer = optim.Adam(discriminator.parameters(), lr=lr)

# Loss function
criterion = nn.BCELoss()

# Training loop (simplified)
def train_gan(epochs=1000):
    for epoch in range(epochs):
        # Train Discriminator
        real_data = generate_real_samples() # Your real data
        fake_data = generator(torch.randn(batch_size, latent_dim))

        d_loss_real = criterion(discriminator(real_data), torch.ones(batch_size, 1))
        d_loss_fake = criterion(discriminator(fake_data.detach()), torch.zeros(batch_size, 1))
        d_loss = d_loss_real + d_loss_fake

        d_optimizer.zero_grad()
        d_loss.backward()
        d_optimizer.step()

        # Train Generator
        fake_data = generator(torch.randn(batch_size, latent_dim))
        g_loss = criterion(discriminator(fake_data), torch.ones(batch_size, 1))

        g_optimizer.zero_grad()
        g_loss.backward()
        g_optimizer.step()

    return d_loss.item(), g_loss.item()

```

1.3 Code Structure for Practice

```

# Complete GAN training pipeline
class GANTrainer:
    def __init__(self, generator, discriminator, device='cpu'):
        self.generator = generator.to(device)
        self.discriminator = discriminator.to(device)
        self.device = device

        # Initialize optimizers
        self.g_optimizer = optim.Adam(generator.parameters(), lr=0.0002, betas=(0.5, 0.999))
        self.d_optimizer = optim.Adam(discriminator.parameters(), lr=0.0002, betas=(0.5, 0.999))

```

```

self.criterion = nn.BCELoss()

def train_step(self, real_data, batch_size, latent_dim):
    # Training step implementation
    device = self.device

    # Labels
    real_labels = torch.ones(batch_size, 1).to(device)
    fake_labels = torch.zeros(batch_size, 1).to(device)

    # Train Discriminator
    self.discriminator.zero_grad()

    # Real data
    real_output = self.discriminator(real_data)
    d_loss_real = self.criterion(real_output, real_labels)

    # Fake data
    noise = torch.randn(batch_size, latent_dim).to(device)
    fake_data = self.generator(noise)
    fake_output = self.discriminator(fake_data.detach())
    d_loss_fake = self.criterion(fake_output, fake_labels)

    d_loss = d_loss_real + d_loss_fake
    d_loss.backward()
    self.d_optimizer.step()

    # Train Generator
    self.generator.zero_grad()
    fake_output = self.discriminator(fake_data)
    g_loss = self.criterion(fake_output, real_labels)
    g_loss.backward()
    self.g_optimizer.step()

    return d_loss.item(), g_loss.item()

```